LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Zidane Abdul Kadir (2509106021)

Kelas (A1 ‘25)

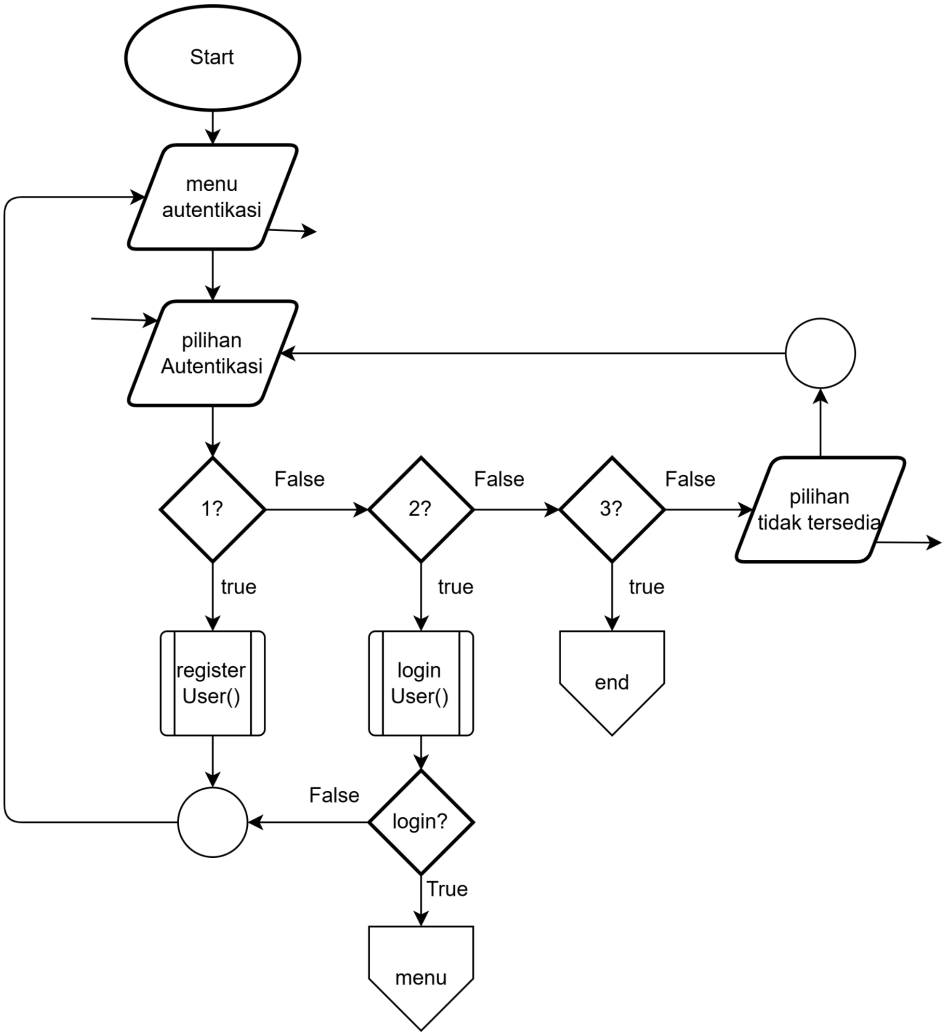
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

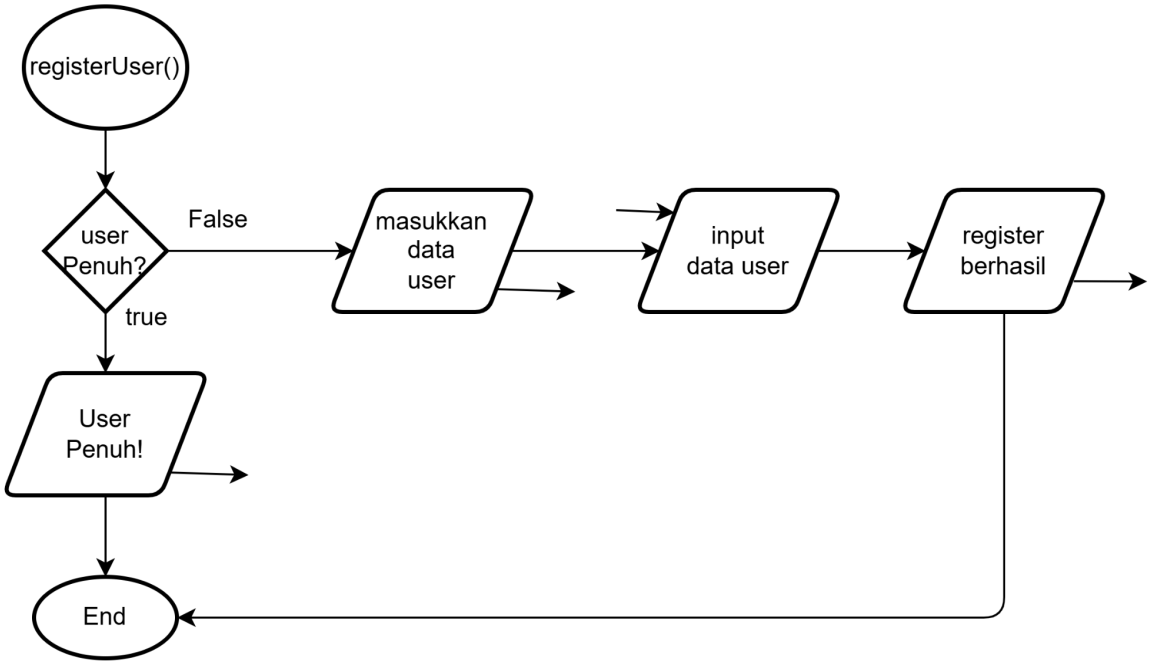
2025

1. Flowchart



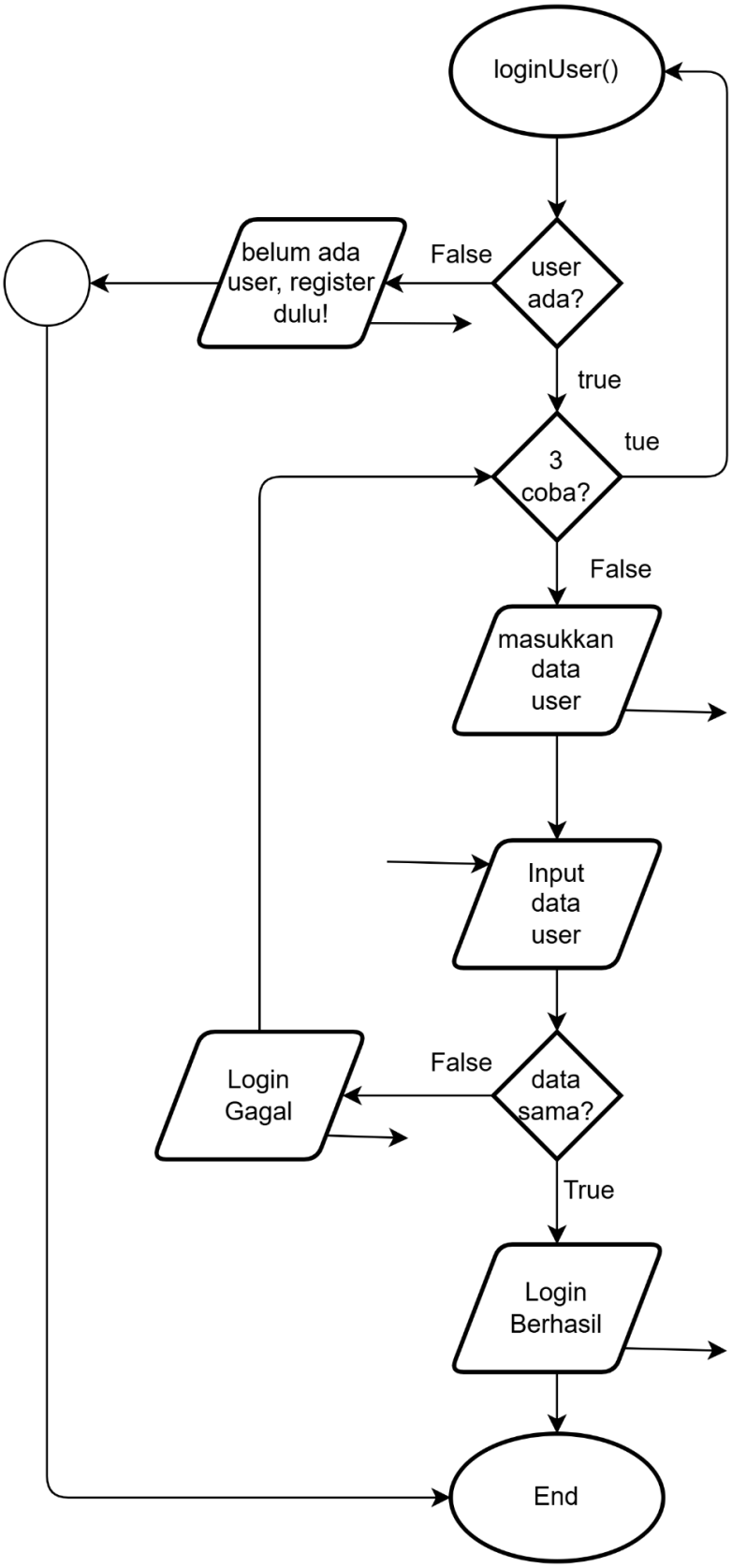
Gambar 1.1

Pada **Gambar 1.1**, diagram alir menjelaskan alur logika awal sistem yang dimulai dengan inisialisasi (Start) dan dilanjutkan dengan penampilan menu autentikasi. Setelah pengguna memberikan input pilihan autentikasi, sistem akan melakukan percabangan kondisi berdasarkan angka yang dimasukkan. Pilihan 1 diarahkan untuk mengeksekusi fungsi register User(), sedangkan pilihan 2 akan menjalankan fungsi login User() yang kemudian divalidasi melalui keputusan 'login?'; jika berhasil, pengguna diarahkan ke menu utama. Pilihan 3 akan mengarahkan program untuk berakhir (end), sementara input yang tidak sesuai akan memicu tampilan pesan 'pilihan tidak tersedia'.



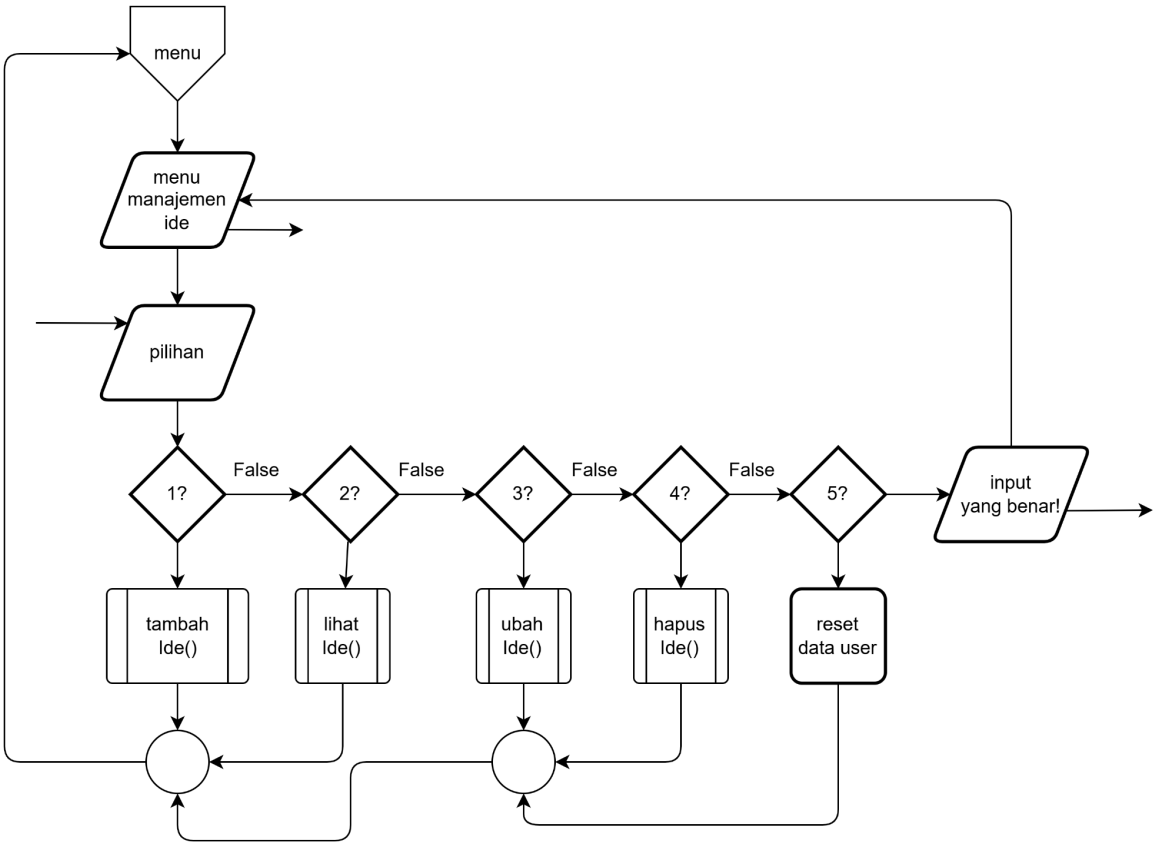
Gambar 1.2

Pada **Gambar 1.2**, diagram alir menguraikan logika prosedur untuk fungsi registerUser() yang menangani pendaftaran akun baru. Proses diawali dengan pengecekan kapasitas penyimpanan melalui keputusan "user Penuh?"; apabila kondisi ini terpenuhi, sistem akan langsung menampilkan pesan peringatan "User Penuh!" dan mengakhiri proses (End). Sebaliknya, jika kapasitas masih tersedia, alur berlanjut ke tahap masukan di mana sistem menampilkan instruksi "masukkan data user", menerima "input data user" dari pengguna, dan diakhiri dengan pesan konfirmasi bahwa "register berhasil".


loginUser()
user ada?
False
belum ada user, register dulu!
true
3 coba?
tue
False
masukkan data user
Input data user
data sama?
False
Login Gagal
True
Login Berhasil
End

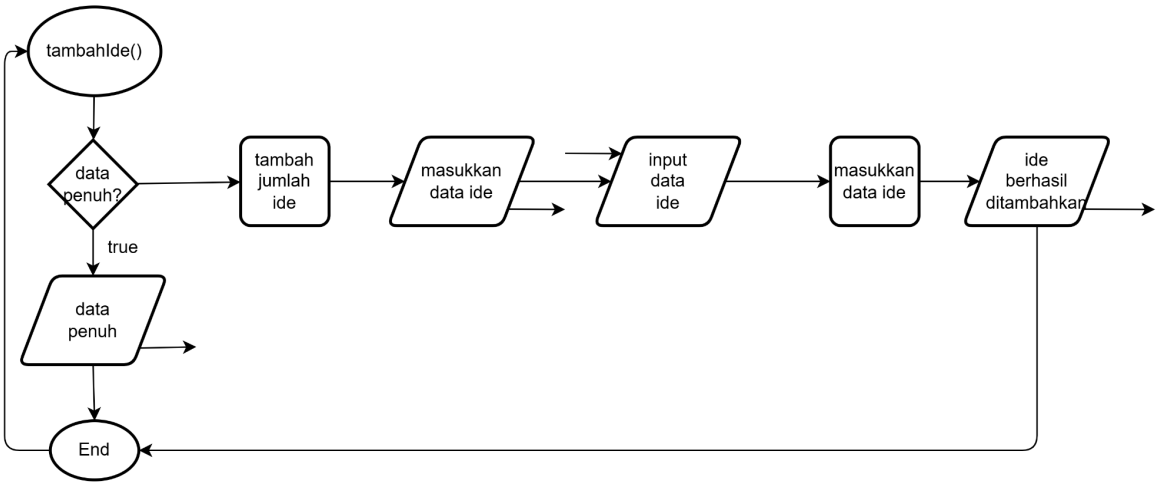
Gambar 1.3

Pada **Gambar 1.3**, diagram alir menguraikan logika prosedur untuk fungsi loginUser(). Proses diawali dengan validasi awal untuk memeriksa apakah terdapat data pengguna ("user ada?"); jika tidak, sistem akan menampilkan peringatan "belum ada user, register dulu!". Apabila data tersedia, alur berlanjut ke proses input data pengguna dengan mempertimbangkan batas percobaan ("3 coba?"). Setelah sistem meminta dan menerima input data, dilakukan pengecekan kesesuaian kredensial ("data sama?"); jika data tidak sesuai, sistem menampilkan pesan "Login Gagal", namun jika data valid, sistem akan memberikan notifikasi "Login Berhasil" sebelum mengakhiri fungsi (End).



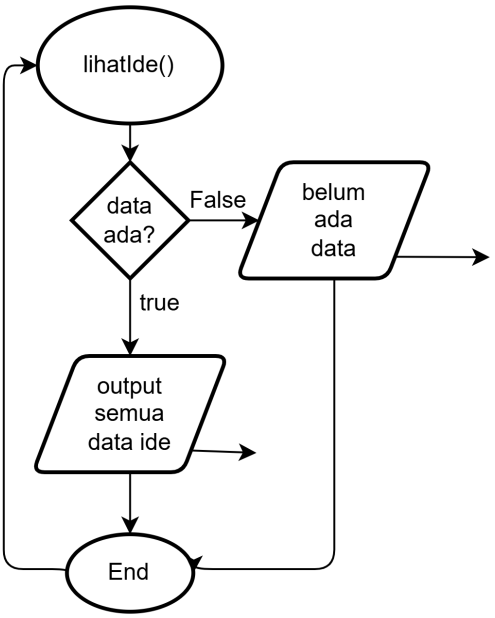
Gambar 1.4

Pada **Gambar 1.4**, diagram alir menjelaskan alur navigasi pada menu manajemen ide yang diakses setelah pengguna berhasil login. Proses dimulai dengan menampilkan daftar opsi fitur ("menu manajemen ide") dan menerima input pilihan dari pengguna. Berdasarkan input tersebut, sistem melakukan percabangan kondisi untuk mengeksekusi fungsi yang sesuai: pilihan 1 memanggil fungsi tambah Ide(), pilihan 2 memanggil fungsi lihat Ide(), pilihan 3 memanggil fungsi ubah Ide(), pilihan 4 memanggil fungsi hapus Ide(), dan pilihan 5 berfungsi untuk melakukan proses reset data user. Selain itu, diagram ini juga mencakup penanganan kesalahan input, di mana jika pilihan yang dimasukkan tidak valid, sistem akan menampilkan pesan "input yang benar!".



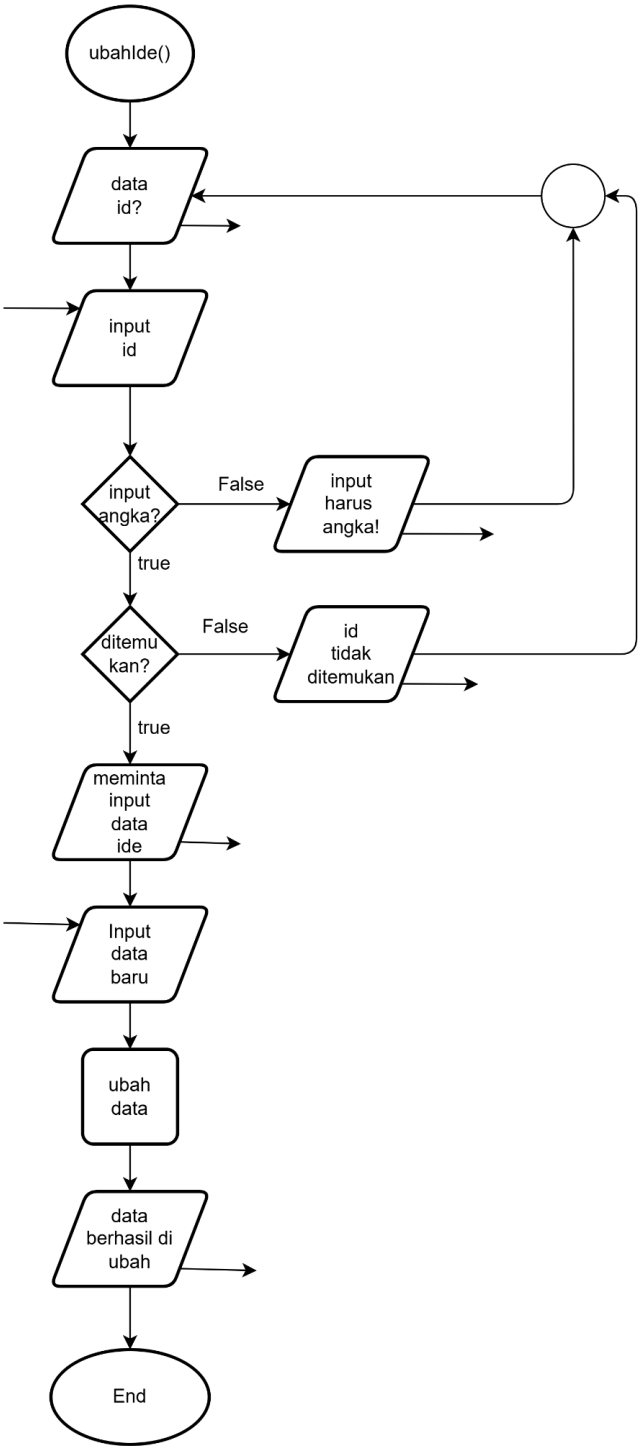
Gambar 1.5

Pada **Gambar 1.5**, diagram alir menjelaskan logika prosedur untuk fungsi tambahIde(). Proses dimulai dengan validasi kapasitas penyimpanan melalui keputusan "data penuh?"; jika kondisi ini terpenuhi, sistem akan langsung menampilkan pesan "data penuh" dan mengakhiri proses. Sebaliknya, jika kapasitas masih tersedia, sistem akan melakukan proses "tambah jumlah ide" untuk memperbarui penghitung, dilanjutkan dengan menampilkan instruksi "masukkan data ide", menerima "input data ide" dari pengguna, memproses penyimpanan data tersebut, dan diakhiri dengan pesan konfirmasi bahwa "ide berhasil ditambahkan".



Gambar 1.6

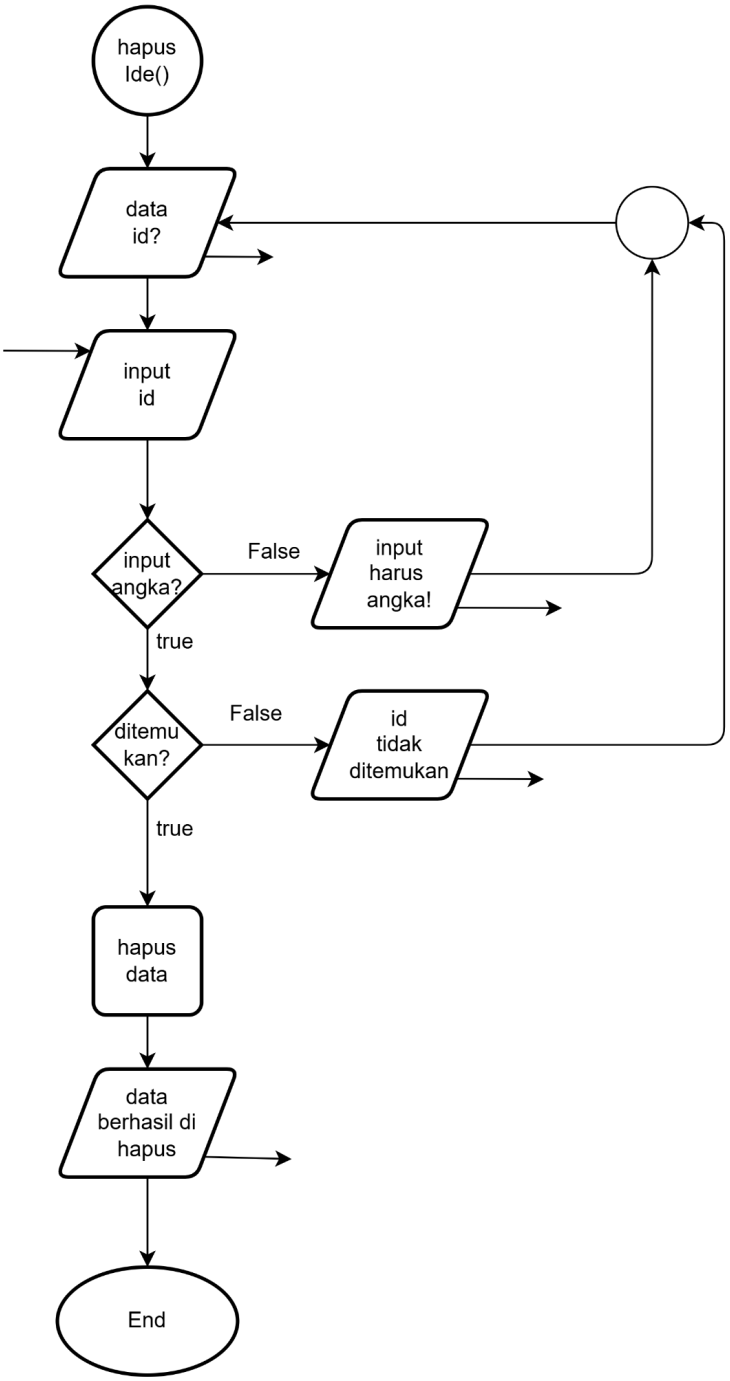
Pada **Gambar 1.6**, diagram alir menguraikan logika prosedur untuk fungsi lihatIde() yang berfungsi untuk menampilkan data ide yang tersimpan. Proses dimulai dengan validasi ketersediaan data melalui keputusan "data ada?"; apabila tidak terdapat data dalam sistem, alur akan mengarah ke proses penampilan pesan "belum ada data". Namun, jika data tersedia, sistem akan melanjutkan ke proses "output semua data ide" untuk menampilkan seluruh informasi ide yang tersimpan dalam format tabel. Setelah proses penampilan data selesai atau jika data kosong, alur diagram akan berakhir (End).



Gambar 1.7

Pada **Gambar 1.7**, diagram alir menguraikan logika prosedur untuk fungsi ubahIde() yang digunakan untuk memperbarui data ide. Proses dimulai dengan meminta pengguna memasukkan identitas data melalui "input id". Sistem kemudian melakukan validasi input dengan mengecek apakah masukan tersebut berupa angka ("input angka?"); jika tidak, sistem akan menampilkan pesan error "input harus angka!". Apabila input valid, sistem melanjutkan dengan pengecekan keberadaan data ("ditemukan?"); jika ID tidak ditemukan, sistem akan menampilkan notifikasi "id tidak ditemukan". Namun, jika data ditemukan, alur berlanjut ke

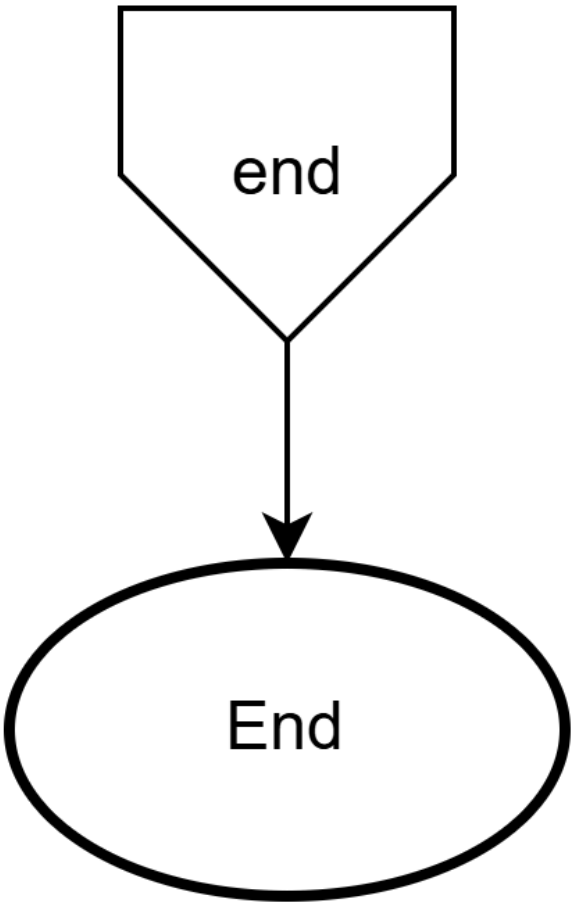
tahap pembaruan di mana sistem meminta "Input data baru", memproses perubahan tersebut ("ubah data"), dan diakhiri dengan pesan konfirmasi "data berhasil di ubah" sebelum fungsi berakhir (End).



Gambar 1.8

Pada **Gambar 1.8**, diagram alir menjelaskan logika prosedur untuk fungsi hapusIde() yang bertujuan untuk menghapus data ide berdasarkan ID. Proses dimulai dengan permintaan identitas data melalui output "data id?" dan penerimaan "input id" dari pengguna. Sistem

kemudian melakukan validasi input dengan mengecek apakah masukan tersebut berupa angka ("input angka?"); jika tidak, sistem akan menampilkan peringatan "input harus angka!". Apabila input valid, sistem melanjutkan pengecekan keberadaan data ("ditemukan?"); jika ID tidak ditemukan, akan muncul notifikasi "id tidak ditemukan". Namun, jika ID ditemukan, sistem akan menjalankan proses "hapus data" dan diakhiri dengan pesan konfirmasi bahwa "data berhasil di hapus" sebelum fungsi berakhir (End).



Gambar 1.9

Pada **Gambar 1.9**, diagram alir menggambarkan tahap terminasi atau penghentian program yang menandakan akhir dari seluruh eksekusi sistem. Flowchart ini menunjukkan konektor "end" yang mengarah langsung ke simbol terminator "End", yang secara visual merepresentasikan titik akhir dari alur logika program. Simbol ini berfungsi sebagai penanda bahwa seluruh rangkaian proses dalam aplikasi telah selesai dijalankan dan program akan berhenti beroperasi secara resmi.

2. Deskripsi Singkat Program

Program ini merupakan aplikasi berbasis console untuk **Manajemen Ide** yang dibangun menggunakan bahasa C++. Sistem ini mengintegrasikan fitur **autentikasi** (Register dan Login) dengan batasan kapasitas maksimal 50 pengguna dan 3 kali percobaan login. Setelah berhasil masuk, pengguna dapat melakukan operasi **CRUD** (Tambah, Lihat, Ubah, Hapus) pada data ide (maksimal 100 data) yang mencakup atribut judul, deskripsi, dan status. Program ini juga dilengkapi dengan validasi input yang ketat untuk mencegah error saat pengguna memasukkan data yang tidak sesuai.

3. Source Code

A. Fitur Register

Fitur ini digunakan untuk mendaftarkan pengguna baru ke dalam sistem. Program akan menyimpan nama dan NIM pengguna ke dalam array daftarUser. Sistem memiliki batasan maksimal 50 pengguna.

Source Code:

```
void registerUser(Sistem *sistem){
    if((*sistem).jumlahUser >= 50){
        tampilkanPesan("User sudah penuh");
        return;
    }

    cout << "Nama: ";
    cin >> (*sistem).daftarUser[(*sistem).jumlahUser].nama;

    cout << "NIM: ";
    cin >> (*sistem).daftarUser[(*sistem).jumlahUser].nim;

    (*sistem).jumlahUser++;
    tampilkanPesan("Register berhasil");
}
```

B. Fitur Login

Fitur ini digunakan untuk autentikasi pengguna yang sudah terdaftar. Sistem memberikan 3 kali percobaan untuk memasukkan nama dan NIM yang sesuai dengan data yang tersimpan.

Source Code:

```
bool loginUser(Sistem *sistem, int percobaan){
    if(percobaan >= 3){
        return false;
    }

    string namaLogin;
    string nimLogin;

    cout << "Nama: ";
    cin >> namaLogin;

    cout << "NIM: ";
    cin >> nimLogin;

    if(cekLoginUser(*sistem, namaLogin, nimLogin)){
        tampilkanPesan("Login berhasil");
        return true;
    }

    tampilkanPesan("Login gagal");
    return loginUser(sistem, percobaan + 1);
}
```

C. Fitur Tambah Ide

Fitur ini memungkinkan pengguna yang sudah login untuk menambahkan ide baru dengan atribut judul, deskripsi, dan status. Sistem memiliki batasan maksimal 100 ide dan ID dibuat otomatis.

Source Code:

```
void tambahIde(Sistem *sistem){
    if((*sistem).jumlahIde >= 100){
        tampilkanPesan("Data penuh");
        return;
    }

    cin.ignore(1000, '\n');
```

```
    (*sistem).daftarIde[(*sistem).jumlahIde].id =
(*sistem).nextIdeId;
    (*sistem).nextIdeId++;

    cout << "Judul Ide: ";
    getline(cin, (*sistem).daftarIde[(*sistem).jumlahIde].judul);

    cout << "Deskripsi Ide: ";
    getline(cin,
(*sistem).daftarIde[(*sistem).jumlahIde].deskripsi);

    (*sistem).daftarIde[(*sistem).jumlahIde].status =
pilihStatusIde();

    (*sistem).jumlahIde++;
    tampilkanPesan("Ide berhasil ditambahkan");
}
```

D. Fitur Lihat Ide

Fitur ini digunakan untuk menampilkan semua ide yang tersimpan dalam sistem dengan format tabel yang rapi menggunakan setw untuk pengaturan lebar kolom.

Source Code:

```
void lihatIde(Sistem *sistem){
    if((*sistem).jumlahIde == 0){
        tampilkanPesan("Belum ada data");
        return;
    }

    cout << left
         << setw(6) << "ID"
         << setw(25) << "Judul"
         << setw(40) << "Deskripsi"
         << setw(15) << "Status" << endl;

    cout << string(86, '-') << endl;

    for(int i = 0; i < (*sistem).jumlahIde; i++){
        cout << left
             << setw(6) << (*sistem).daftarIde[i].id
```

```
            << setw(25) << (*sistem).daftarIde[i].judul
            << setw(40) << (*sistem).daftarIde[i].deskripsi
            << setw(15) << (*sistem).daftarIde[i].status
            << endl;
    }
}
```

E. Fitur Ubah ide

Fitur ini memungkinkan pengguna untuk mengubah data ide yang sudah ada berdasarkan ID. Pengguna dapat mengubah judul, deskripsi, dan status ide.

Source Code :

```
void ubahIde(Sistem *sistem){
    int id = inputAngka("Masukkan ID: ");
    int indeks = cariIdeById(sistem, id, 0);

    if(indeks == -1){
        tampilkanPesan("ID tidak ditemukan");
        return;
    }

    cin.ignore(1000, '\n');

    cout << "Judul baru: ";
    getline(cin, (*sistem).daftarIde[indeks].judul);

    cout << "Deskripsi baru: ";
    getline(cin, (*sistem).daftarIde[indeks].deskripsi);

    (*sistem).daftarIde[indeks].status = pilihStatusIde();

    tampilkanPesan("Data berhasil diubah");
}
```

F. Fitur Hapus Ide

Fitur ini digunakan untuk menghapus data ide berdasarkan ID. Setelah data dihapus, seluruh data setelahnya akan bergeser untuk menutupi posisi data yang dihapus.

Source Code:

```
void hapusIde(Sistem *sistem){
    int id = inputAngka("Masukkan ID: ");
    int indeks = cariIdeById(sistem, id, 0);
```

```
    if(indeks == -1){
        tampilkanPesan("ID tidak ditemukan");
        return;
    }

    for(int i = indeks; i < (*sistem).jumlahIde - 1; i++){
        (*sistem).daftarIde[i] = (*sistem).daftarIde[i + 1];
    }

    (*sistem).jumlahIde--;
    tampilkanPesan("Data berhasil dihapus");
}
```

4. Hasil Output

```
===== MENU AWAL =====
1 Register
2 Login
3 Keluar Program
Pilihan: 1
Nama: zidan
NIM: 021
Register berhasil
===== MENU AWAL =====
1 Register
2 Login
3 Keluar Program
Pilihan: 2
Nama: zidan
NIM: 021
Login berhasil

===== MENU Manajemen Ide =====
1 Tambah Ide
2 Lihat Ide
3 Ubah Ide
4 Hapus Ide
5 Logout
Pilihan:
```

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git add .
```

menambahkan perubahan pada repositori

5.2 GIT Commit

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git commit -m "PT-2"
[main c8c7fbf] PT-2
 3 files changed, 967 insertions(+), 9 deletions(-)
 create mode 100644 flowchart/PT-2.drawio
 create mode 100644 post-test/post-test-apl-2/2509106021-MuhammadZidaneAbdulKadir-PT-2.exe
```

melakukan commit pada repositori

5.3 GIT Push

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 28.85 KiB | 2.22 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ExtraYiban/praktikum-apl.git
   c37df2f..c8c7fbf  main -> main
```

mengiriim commit ke github